MAINTENANCE GUIDE

Guía de Mantenimiento

Emulador VASSAL para OCS en iPad

> Repositorio: github.com/JaviARQ/vassal-ocs
>
> Versión: 2.0 · Noviembre 2026
>
> Modo: clean repo + load .vmod from file

Índice

1. Introducción y contexto del proyecto.....

1. Introducción y contexto del proyecto.....1

1.1 Filosofía de uso del proyecto.....1

1.2 Estructura del proyecto local.....2

1.3 URLs y datos clave.....5

2. Sincronización con el repositorio original.....5

2.1 Concepto: origin vs upstream.....5

2.2 Comprobación del remote upstream.....5

2.3 Rutina de sincronización.....7

2.4 Resolución de conflictos.....9

2.5 Archivos sensibles a conflictos.....11

2.6 Cuándo y cómo upstream nuestros parches.....12

3. Carga de módulos .vmod.....14

3.1 Opción A — Carga desde archivo local (workflow principal).....14

Pasos en el iPad.....15

Aviso sobre clases Java personalizadas.....15

3.2 Opción B — Presets empaquetados en el repo (alternativa).....16

Pasos.....16

Comandos.....17

3.3 Guardar y cargar partidas (.vsav).....19

Guardar partida.....19

Cargar partida.....20

4. Uso offline desde el iPad.....20

4.1 Concepto del service worker.....20

4.2 Instalación inicial en el iPad (una sola vez por dispositivo).....21

4.3 Flujo de juego offline.....22

4.4 Cómo forzar refresco si la app no se actualiza.....23

4.5 Cómo resetear la caché del SW por completo....23
En el PC (Chrome/Edge)....24
En el iPad....24
4.6 Consideraciones sobre el almacenamiento del iPad....25
5. Otras actividades de mantenimiento....25
5.1 Verificación del workflow GitHub Actions....26
5.2 Renovación del Personal Access Token (PAT)....26
Renovación....26
Actualizar en Windows....27
5.3 Bump del cache version del service worker....28
5.4 Backup del archivo presets.json....30
5.5 Limpieza del .gitignore....30
5.6 Reportar bugs al autor original....32
5.7 Probar módulos nuevos antes de usarlos en serio....33
6. Troubleshooting....34
6.1 Errores de git....34
6.2 Errores de npm....34
6.3 Errores del workflow GitHub Actions....34
6.4 Errores de carga del módulo en el emulador....34
6.5 Errores al guardar/cargar .vsav....35
6.6 Errores en el iPad....35
7. Cheat sheet — comandos rápidos....35
7.1 Setup inicial (una sola vez)....35
7.2 Rutinas habituales....35
7.3 URLs importantes....36
7.4 Archivos clave del proyecto....36
7.5 Flujo de juego offline (resumen)....36
8. Glosario....36
8.1 Git y GitHub....37
8.2 Node.js, npm y Vite....37

8.3 VASSAL y OCS.....37

8.4 Service worker y PWA.....37

8.5 GitHub Pages y GitHub Actions.....37

Nota: este índice se genera mediante campos. Para actualizar los números de página tras editar, haz clic derecho sobre el índice y selecciona Actualizar campo.

1. Introducción y contexto del proyecto

Este documento recoge las rutinas de mantenimiento del proyecto vassal-ocs, una bifurcación del emulador TypeScript de VASSAL publicada por TyroneSlothrop en GitHub. El proyecto está desplegado en GitHub Pages y permite jugar módulos VASSAL —en particular los de la serie OCS (Operational Combat Series)— offline desde un iPad sin necesidad de Java ni de servidor dedicado.

La guía se redacta como referencia viva: está pensada para consultarse cada vez que se quiera actualizar el código desde el repositorio original, gestionar el service worker del iPad o diagnosticar un problema. Cada sección incluye los comandos exactos a ejecutar en una ventana de símbolo del sistema (cmd) de Windows, los resultados esperados y los errores típicos con su solución.

1.1 Filosofía de uso del proyecto

El repositorio está configurado en modo clean: no incluye módulos .vmod empaquetados. El emulador se sirve desde GitHub Pages y los módulos se cargan al vuelo desde la app Archivos del iPad, mediante el botón Load module from file del propio emulador. Esto tiene varias implicaciones:

- **Repo ligero:** el repo solo contiene el código del emulador (~1 MB), no .vmod cargados.
- **Sin preocupaciones de espacio:** no hay que vigilar el tamaño del repo ni de los archivos.
- **Workflow simple:** descargar .vmod desde vassalengine.org directamente en el iPad y cargarlo en el emulador.
- **Cualquier módulo OCS:** se puede jugar cualquier título sin tener que subirlo antes al repo.
- **Offline funcional:** el emulador está cacheado por el service worker; el .vmod vive en la app Archivos del iPad. No se necesita red para jugar.

1.2 Estructura del proyecto local

El proyecto se encuentra en C:\Users\JaviPC\vassal-ocs. La estructura interna relevante para el mantenimiento es la siguiente:

```
vassal-ocs/
├── .github/workflows/
│   └── deploy.yml          ← workflow de GitHub Actions
├── .gitignore              ← creado por nosotros (excluye
node_modules, dist, etc.)
├── index.html              ← registra el service worker
├── package.json            ← dependencias y scripts npm
├── package-lock.json
├── vite.config.ts          ← base: "./" para servir en
subdirectorio
├── tsconfig.json
├── scripts/
```

```
│   └── gen-presets.mjs      ← PARCHEADO: usa fileURLToPath (fix
Windows)
├── src/
│   └── app.ts               ← PARCHEADO: filtro accept del .vsav
eliminado (fix iOS)
├── public/
│   ├── sw.js                ← service worker (offline)
│   ├── sw-register.js       ← registro del SW desde index.html
│   └── modules/
│       └── presets.json     ← vacío por defecto ([])
└── dist/                    ← generada por npm run build (no se
commitea)
```

1.3 URLs y datos clave

Concepto	Valor
Repositorio propio (origin)	https://github.com/JaviARQ/vassal-ocs
Sitio publicado (Pages)	https://javiarq.github.io/vassal-ocs/
Repositorio original (upstream)	https://github.com/TyroneSlothrop/vassal-ts-emulator
Ruta local del proyecto	C:\Users\JaviPC\vassal-ocs
Usuario GitHub	JaviARQ
Branch principal	main
Workflow de deploy	.github/workflows/deploy.yml
Service worker	public/sw.js (cache: vassal-ocs-v1)
Biblioteca de módulos VASSAL	https://vassalengine.org/library

2. Sincronización con el repositorio original

El emulador lo mantiene activamente TyroneSlothrop. Cuando el autor publica mejoras (nuevos traits soportados, corrección de bugs, optimizaciones para iOS), interesa absorber esos cambios en nuestra bifurcación sin perder los añadidos propios: parche Windows de gen-presets.mjs, parche iOS Safari de src/app.ts, .gitignore y service worker.

2.1 Concepto: origin vs upstream

Git permite tener varios remotes (punteros a repositorios remotos). Por defecto, al clonar un repo se crea el remote origin apuntando al repo del que se clonó. Para seguir la evolución del original, se añade un segundo remote llamado upstream.

Remote	URL	Para qué sirve
origin	https://github.com/JaviARQ/vassal-ocs.git	Es TU repo. Aquí empujas tus cambios.
upstream	https://github.com/TyroneSlothrop/vassal-ts-emulator.git	Es el original. Solo se lee (fetch) para traerse novedades.

2.2 Comprobación del remote upstream

Para verificar que el remote upstream sigue configurado (se configuró una sola vez al inicio del proyecto):

```
cd %USERPROFILE%\vassal-ocs
git remote -v
```

Deben aparecer cuatro líneas, dos por cada remote (fetch y push):

```
origin    https://github.com/JaviARQ/vassal-ocs.git (fetch)
origin    https://github.com/JaviARQ/vassal-ocs.git (push)
upstream  https://github.com/TyroneSlothrop/vassal-ts-emulator.git
(fetch)
upstream  https://github.com/TyroneSlothrop/vassal-ts-emulator.git
(push)
```

Si upstream no aparece, se añade con:

```
git remote add upstream https://github.com/TyroneSlothrop/vassal-ts-
emulator.git
```

2.3 Rutina de sincronización

Antes de ejecutar la rutina, conviene echar un vistazo al historial del original para ver si hay cambios que merezcan la pena:

Visita `https://github.com/TyroneSlothrop/vassal-ts-emulator/commits/main` y mira los commits recientes. Si no hay nada nuevo desde la última sincronización, no hace falta hacer nada.

La rutina completa, ejecutar en cmd dentro del proyecto:

```
cd %USERPROFILE%\vassal-ocs

REM 1. Traerse los cambios del original (no modifica tus archivos)
git fetch upstream

REM 2. (Opcional) Ver qué commits hay nuevos que aún no tienes
git log HEAD..upstream/main --oneline

REM 3. Fusionar upstream/main con tu main
```

```
git merge upstream/main

REM 4. Si no hubo conflictos: empujar a tu repo
git push
```

Tras el git push, el workflow de GitHub Actions se dispara solo y reconstruye el sitio con la nueva versión del emulador.

> **Buena práctica: prueba en local antes de pushear**
>
> Si el merge trae muchos cambios (especialmente en código TypeScript del emulador), es recomendable probar antes en local con npm run dev y cargar un módulo para confirmar que nada se ha roto. Solo cuando funciona se hace el push.

2.4 Resolución de conflictos

Un conflicto ocurre cuando upstream modifica un archivo que tú también has modificado. En este proyecto hay dos archivos sensibles: scripts/gen-presets.mjs (parche Windows fileURLToPath) y src/app.ts (parche iOS para .vsav).

Si al ejecutar git merge upstream/main aparece un mensaje como:

```
CONFLICT (content): Merge conflict in scripts/gen-presets.mjs
```

Git detiene el merge y marca el archivo con tres bloques delimitados por signos:

```
<<<<<<< HEAD
(tu versión local con el parche)
=======
(versión nueva que viene de upstream)
>>>>>>> upstream/main
```

Pasos para resolverlo:

- **Abrir el archivo** con el Bloc de notas o tu editor favorito.
- **Localizar los tres bloques** <<<<<<< HEAD, ======= y >>>>>>> upstream/main.
- **Decidir la versión final:** mantener nuestro parche más cualquier mejora nueva del autor que sea compatible. Si el autor hubiera corregido él mismo el bug, su versión ya sería buena y podríamos eliminar la nuestra.
- **Borrar las tres marcas** <<<<<<<, ======= y >>>>>>> y dejar el código final limpio.
- **Guardar el archivo** y en la cmd ejecutar:

```
git add <archivo-resuelto>
git commit -m "merge upstream + mantener parches locales"
git push
```

Abortar un merge si te pierdes

Si la resolución del conflicto se te complica, SIEMPRE puedes abortar y volver al estado anterior con: git merge --abort. Esto deja todo como estaba antes de empezar el merge, sin pérdida de datos.

2.5 Archivos sensibles a conflictos

Archivo	Parche local	Riesgo de conflicto
scripts/gen-presets.mjs	fileURLToPath en vez de .pathname	Medio: bug Windows bien conocido, el autor podría arreglarlo
src/app.ts	Comentar input.accept = ".vsav,..." en pickAndLoadGame()	Medio: bug iOS Safari bien conocido
.gitignore	Creado desde cero (no existía en upstream)	Bajo: si upstream añade su propio .gitignore, conflicto fácil de resolver
.github/workflows/deploy.yml	Creado desde cero	Bajo: si upstream añade su propio workflow
public/sw.js + sw-register.js	Creados desde cero	Bajo: si upstream añade SW propio

2.6 Cuándo y cómo upstream nuestros parches

Los dos parches que aplicamos (fileURLToPath para Windows y eliminación del filtro accept para iOS Safari) son bug fixes reales que afectan a cualquier usuario Windows e iOS del proyecto original, no algo específico de nuestro setup. Lo ideal sería enviarlos al autor original como Pull Request (PR). Si los acepta:

- A partir de ese momento, upstream ya lleva los parches.
- Nuestros futuros merges no tendrán conflicto en esos archivos.
- Otros usuarios Windows e iOS se benefician.

Pasos para abrir un PR (operación opcional, hacer una sola vez):

```
REM En GitHub web: haz fork del original desde
REM https://github.com/TyroneSlothrop/vassal-ts-emulator
REM (botón Fork arriba a la derecha)

REM Añade tu fork como otro remote (lo llamamos "myfork")
git remote add myfork https://github.com/JaviARQ/vassal-ts-
emulator.git
```

```
REM Crea una rama específica para el PR
git checkout -b fix/windows-and-ios-patches

REM Asegúrate de que en esa rama solo están los parches
REM (puedes cherry-pick los commits originales o crear nuevos)

git push -u myfork fix/windows-and-ios-patches
```

Tras ese push, ve a tu fork en GitHub (https://github.com/JaviARQ/vassal-ts-emulator) y aparecerá un banner verde con un botón Compare & pull request. Sigue el asistente, escribe un título claro como fix: Windows path bug in gen-presets + iOS Safari .vsav accept filter, y una descripción explicando ambos bugs. El autor recibirá el PR y decidirá si lo mergea.

3. Carga de módulos .vmod

El emulador admite dos formas de cargar módulos. La principal y recomendada para uso personal desde el iPad es la carga desde archivo local. La carga vía presets empaquetados en el repo queda como opción alternativa si en algún momento se quiere tener un módulo siempre disponible en el selector.

3.1 Opción A — Carga desde archivo local (workflow principal)

Este es el flujo recomendado para uso personal. No requiere tocar el repo, no añade peso al repo, no requiere deploy. Funciona para cualquier módulo .vmod de cualquier serie, no solo OCS.

Pasos en el iPad

- **1. Descargar el .vmod:** en Safari del iPad, ve a https://vassalengine.org/library, busca el módulo y descarga la última versión. El archivo se guarda en la app Archivos → Descargas.
- **2. Abrir el emulador:** abre la app VASSAL desde el icono de pantalla de inicio (previamente añadido con Compartir → Añadir a pantalla de inicio).
- **3. Cargar el módulo:** pulsa el botón Load module from file del emulador. Se abre el explorador de archivos del iPad.
- **4. Seleccionar el .vmod:** navega a Descargas (o donde lo hayas guardado) y selecciona el archivo .vmod. El emulador lo carga y muestra el mapa y las piezas.
- **5. Jugar:** ya puedes jugar. Para guardar la partida, botón Save (baja un .vsav a Descargas). Para cargar partida guardada, botón Load partida y selecciona el .vsav.

Offline con este workflow

Una vez cargado el módulo desde archivo, puedes seguir jugando offline aunque cierres y abras la app, porque el .vmod sigue en la app Archivos del iPad y el emulador está cacheado por el service worker. Solo necesitas red la primera vez (para registrar el SW) y luego puedes jugar sin WiFi.

Aviso sobre clases Java personalizadas

El emulador interpreta el formato .vmod pero no ejecuta clases Java personalizadas. Módulos OCS modernos (Tunisia II desde v1.0, y la mayoría de los publicados después de 2020) están limpios de Java y funcionan bien. Módulos antiguos pueden tener Java dependiente que no funcionará. Si un módulo no carga bien, mira la consola F12 del navegador para ver el error: lo más probable es que un trait o una pieza dependan de Java.

3.2 Opción B — Presets empaquetados en el repo (alternativa)

Si en algún momento quieres tener uno o varios módulos siempre pre-cargados en el selector del emulador (sin tener que cargarlos desde archivo cada vez), puedes empaquetarlos en el repo siguiendo este procedimiento.

Solo si lo necesitas

Esta opción NO es necesaria para el uso normal. Solo la activas si quieres tener un módulo siempre disponible en el selector del emulador, o si quieres compartir el repo con otra persona que no tenga los .vmod descargados.

Pasos

- **1. Descargar el .vmod** a tu PC desde vassalengine.org.
- **2. Crear la carpeta del módulo** dentro de public/modules/ con un nombre corto en minúsculas (p.ej. tunisia-ii).
- **3. Copiar el .vmod** a esa carpeta.
- **4. Regenerar presets.json:** ejecuta npm run presets dentro del proyecto.
- **5. Probar en local:** npm run dev y carga el módulo en http://localhost:5173.
- **6. Si funciona:** git add ., git commit, git push.
- **7. Esperar al deploy:** GitHub Actions reconstruye el sitio en ~1-2 min.

Comandos

```
cd %USERPROFILE%\vassal-ocs

REM Crear carpeta del módulo y copiar el .vmod
```

```
mkdir public\modules\<nombre>
copy "%USERPROFILE%\Downloads\<archivo>.vmod"
public\modules\<nombre>\

REM Regenerar presets
npm run presets

REM Probar en local
npm run dev

REM Cuando funcione, subir
git add .
git commit -m "feat: añadir <nombre del módulo> <versión>"
git push
```

Verifica siempre el git status antes de committear

Tras git add, ejecuta git status. Si aparece cualquier cosa de node_modules/, aborta con git reset y revisa el .gitignore. El commit solo debe llevar la carpeta nueva del módulo y presets.json actualizado.

3.3 Guardar y cargar partidas (.vsav)

El emulador admite guardar y cargar partidas en formato .vsav, compatible con VASSAL Java de escritorio. Los archivos se guardan en la app Archivos del iPad (carpeta Descargas) y se cargan desde allí.

Guardar partida

- **Botón Save** del emulador. Genera un archivo .vsav que se descarga a la app Archivos → Descargas.
- **Nombre del archivo:** por defecto lleva el nombre del módulo y la fecha. Puedes renombrarlo desde la app Archivos si quieres.

Cargar partida

- **Botón Load partida** del emulador. Abre el explorador de archivos del iPad.
- **Navega a Descargas** y selecciona el .vsav que quieres cargar.
- **Importante:** el módulo al que pertenece la partida debe estar cargado antes de intentar cargar el .vsav. Si no, el emulador no sabrá interpretar las piezas del save.

Parche iOS Safari aplicado

En el emulador original, el selector de archivos para cargar .vsav tenía un filtro accept=".vsav,..." que hacía que iOS pusiera el archivo en gris al no reconocer la extensión. Este fork elimina ese filtro, por lo que los .vsav se pueden seleccionar directamente sin tener que renombrarlos a .zip.

4. Uso offline desde el iPad

El servicio de cache offline se basa en un service worker (SW), un script JavaScript que el navegador instala la primera vez que visita el sitio y que a partir de entonces intercepta todas las peticiones de red: si hay conexión, sirve la versión más reciente y la cachea; si no hay, sirve la versión cacheada. Esto permite usar el emulador en el iPad aunque no haya WiFi.

4.1 Concepto del service worker

El SW del proyecto (public/sw.js) usa dos estrategias según el tipo de recurso:

Tipo de recurso	Estrategia	Comportamiento
HTML, JS, CSS (interfaz del emulador)	Network-first	Siempre intenta red primero (versión nueva). Si no hay red, sirve cache.
.vmod, imágenes, fuentes	Stale-while-revalidate	Sirve cache al instante (rápido) y descarga la versión nueva en segundo plano para la próxima visita.

Además, el SW está configurado con self.skipWaiting() y self.clients.claim(), lo que significa que cuando se publica una versión nueva del SW, este se activa inmediatamente sin esperar a que se cierren todas las pestañas.

4.2 Instalación inicial en el iPad (una sola vez por dispositivo)

Para que el offline funcione, el iPad tiene que visitar el sitio online al menos una vez para que el SW se registre y cachee los assets del emulador. Pasos:

- **1. iPad online** (WiFi).
- **2. Abre Safari** (importante: Safari, no Chrome) y ve a https://javiarq.github.io/vassal-ocs/ (minúsculas).
- **3. Espera a que cargue** la pantalla del emulador (un canvas con menús arriba).
- **4. Añade a pantalla de inicio:** botón Compartir (cuadrado con flecha arriba) → Añadir a pantalla de inicio → nombre sugerido VASSAL OCS → Añadir.
- **5. Cierra Safari del todo.**

Crítico: la URL va en minúsculas

La URL debe ser https://javiarq.github.io/vassal-ocs/ con vassal-ocs todo en minúsculas. Si pones Vassal-ocs (con V mayúscula), GitHub Pages devuelve 404 porque distingue mayúsculas de minúsculas en el path.

4.3 Flujo de juego offline

Una vez instalado el SW y añadido el icono, el flujo para jugar offline es:

- **1. Modo avión ON** (o sin WiFi, lo que sea).
- **2. Abre la app VASSAL** desde el icono de pantalla de inicio (no desde Safari).
- **3. Botón Load module from file** → explorador → Descargas → selecciona el .vmod que tengas descargado.
- **4. El módulo carga** y puedes jugar sin red.
- **5. Para guardar partida:** botón Save → el .vsav se guarda en Descargas.
- **6. Para cargar partida:** botón Load partida → selecciona el .vsav (con el parche aplicado, ya deja seleccionarlo sin renombrar).

¿De dónde sale el .vmod si estoy offline?

El .vmod se descargó previamente (cuando había red) desde vassalengine.org en Safari del iPad. Se guardó en la app Archivos → Descargas. Una vez ahí, vive en el iPad y no necesita red para abrirse. Por eso el flujo offline funciona: el emulador está cacheado por el SW y el .vmod está en almacenamiento local del iPad.

4.4 Cómo forzar refresco si la app no se actualiza

Si tras un deploy nuevo el iPad sigue mostrando la versión vieja del emulador, prueba en orden:

- **1. Cierra Safari del todo** (desliza desde abajo, desliza Safari hacia arriba).
- **2. Abre Safari, ve a la URL** y espera 5 segundos con la página visible. El SW nuevo (si lo hay) se descarga y se activa en segundo plano.
- **3. Recarga** deslizando hacia abajo desde la parte superior de la página (pull to refresh).
- **4. Si sigue sin actualizarse:** ajustes de Safari → Safari → Avanzado → Datos de sitios web → busca javiarq.github.io y bórralo. Vuelve a visitar la URL online.

4.5 Cómo resetear la caché del SW por completo

Si la caché se ha corrompido o hay un asset a medias que impide cargar el emulador, se puede hacer un reset completo. Esto borra todo lo cacheado y obliga a una descarga limpia desde cero:

En el PC (Chrome/Edge)

```
F12 → Application → Service Workers
  → pulsa "Unregister" en el SW de vassal-ocs
F12 → Application → Storage
  → pulsa "Clear site data"
```

En el iPad

iOS no ofrece una interfaz directa para gestionar SW por sitio. El procedimiento es:

- **Ajustes → Safari → Avanzado → Datos de sitios web** → busca javiarq.github.io → Editar → Eliminar.
- **Abre Safari** y vuelve a visitar la URL online. Se reinstala el SW desde cero.
- **Vuelve a añadir a pantalla de inicio** (el icono anterior puede seguir apuntando a la caché vieja).

4.6 Consideraciones sobre el almacenamiento del iPad

Cada .vmod que descargas ocupa entre 15 y 70 MB en la app Archivos. Si juegas a muchos módulos distintos, conviene revisar de vez en cuando la carpeta Descargas y borrar los .vmod que ya no uses. Lo mismo con los .vsav: cada partida guardada es un archivo de unos pocos KB, pero se acumulan con el tiempo.

Para gestionar el almacenamiento: app Archivos → Descargas → mantén pulsado sobre un archivo → Eliminar. O ajustes → General → Almacenamiento del iPad → Safari → Revisar archivos descargados.

5. Otras actividades de mantenimiento

Además de sincronizar con upstream y cargar módulos, hay varias tareas menores que conviene realizar de vez en cuando para mantener el proyecto sano.

5.1 Verificación del workflow GitHub Actions

Cada vez que se hace un push, el workflow se dispara. Es buena práctica abrir la pestaña Actions tras cada push y confirmar que se pone verde. Si se queda en rojo, hay que investigar inmediatamente antes de seguir añadiendo cambios (ver capítulo 6 para troubleshooting).

URL de monitorización: https://github.com/JaviARQ/vassal-ocs/actions

También se puede ver el historial: cada run se guarda con su commit asociado, duración y estado. Útil para detectar si un deploy empezó a fallar a partir de un commit concreto.

5.2 Renovación del Personal Access Token (PAT)

El PAT creado para autenticar los push tiene una cadencia (lo normal es 90 días). Cuando expira, los push fallan con un error 403 o Authentication failed.

Comprobación previa: ve a https://github.com/settings/tokens y mira la columna Expires. Si está en amarillo o rojo, renueva ya.

Renovación

- Ve a https://github.com/settings/tokens
- Localiza el token vassal-ocs deploy.
- Click en Regenerate token (regenera con la misma cadencia) o elimínalo y crea uno nuevo.
- Copia el token nuevo.

Actualizar en Windows

Windows guarda las credenciales en el Gestor de credenciales. Hay dos formas de actualizarlas:

- **Opción A — Automática:** el primer push que falle te abrirá una ventana emergente de Git Credential Manager; vuelve a autorizar y listo.
- **Opción B — Manual:** abre Administrador de credenciales de Windows (Panel de control → Cuentas de usuario → Administrador de credenciales) → Credenciales de Windows → busca la entrada git:https://github.com → Eliminar. El próximo push te pedirá autenticarte de nuevo.

> **Configuración del PAT para que no caduque tan a menudo**
>
> Al crear el PAT, en Expiration puedes elegir No expiration (sin caducidad). GitHub muestra un aviso de seguridad porque es menos seguro, pero para un repo de hobby personal es razonable. Si lo prefieres más seguro, deja 90 días y renueva cuando toque.

5.3 Bump del cache version del service worker

El service worker tiene una constante CACHE al principio del archivo public/sw.js. Actualmente vale vassal-ocs-v1. Cuando se cambia el valor, todos los dispositivos que visiten el sitio harán una recarga completa (cache vieja borrada, nueva descargada).

Cuándo hacer bump:

- Si has hecho cambios sustanciales y quieres asegurar que todos los dispositivos ven la versión nueva sin depender de expiraciones de caché HTTP.
- Si sospechas que algún asset se ha cacheado mal y está causando errores.

Cómo hacerlo:

```
REM Abre public/sw.js con el Bloc de notas
REM Cambia la línea:
REM   const CACHE = 'vassal-ocs-v1';
```

```
REM por:
REM   const CACHE = 'vassal-ocs-v2';

REM Commit y push
git add public/sw.js
git commit -m "chore: bump service worker cache version"
git push
```

> **Tras un bump, todos los dispositivos tienen que recargar todo online**
>
> Si tienes varios dispositivos con el emulador instalado, después de un bump cada uno tiene que volver a visitar el sitio online para reinstalar el SW y cachear los assets de nuevo. Úsalo solo cuando sea necesario.

5.4 Backup del archivo presets.json

Si tienes módulos empaquetados en presets (Opción B del capítulo 3), public/modules/presets.json es generado por npm run presets pero puede contener metadatos editados a mano. Conviene hacer una copia de seguridad periódica:

```
copy %USERPROFILE%\vassal-ocs\public\modules\presets.json
%USERPROFILE%\Documents\vassal-ocs-presets-backup.json
```

OJO: este backup es solo local. Como el archivo ya está commiteado en git, la mejor copia de seguridad es el propio historial de GitHub. Si pierdes el archivo local, basta con git pull para recuperarlo.

5.5 Limpieza del .gitignore

Cada cierto tiempo, revisa el .gitignore y añade cualquier nuevo patrón de archivos que no deban subirse. Patrones típicos a verificar:

```
node_modules/
dist/
*.log
npm-debug.log*
.vscode/
.idea/
.DS_Store
Thumbs.db
*.local
*.bak
```

Si en algún momento el editor (VS Code, Notepad++) crea archivos temporales que se cuelan en git status, añade su patrón al .gitignore y vuelve a hacer git add -A.

5.6 Reportar bugs al autor original

Si al probar un módulo descubres un bug del emulador (no del módulo), es buena práctica reportarlo al autor original en https://github.com/TyroneSlothrop/vassal-ts-emulator/issues. Información útil para incluir:

- Módulo concreto y versión (p.ej. Tunisia II 1.17.2).
- Qué acción intentabas (arrastrar pieza, abrir paleta, lanzar dado…).
- Mensaje de error exacto de la consola (F12 → Console).
- Screenshot del estado del tablero cuando falla.
- Tu navegador y sistema (Chrome 130 en Windows 11, Safari en iPadOS 17…).

El autor mantiene el proyecto activamente y responde razonablemente rápido. Reportar bugs ayuda a mejorar el emulador para todos.

5.7 Probar módulos nuevos antes de usarlos en serio

Aunque el flujo de carga desde archivo local no requiere subir nada al repo, conviene probar cada módulo nuevo antes de empezar una partida seria. La prueba mínima consiste en:

- Cargar el módulo sin errores en la consola (F12 si estás en PC).
- Ver el mapa / tablero.
- Ver la paleta de piezas.
- Mover una pieza del palette al mapa.
- Seleccionar una pieza en el mapa.
- Probar un botón de menú (fase, dado, etc.).

Si todo esto funciona, el módulo está listo para jugar. Si alguno falla, diagnostica antes de empezar una partida seria (podrías perderte el estado si el emulador casca a mitad.

6. Troubleshooting

Recopilación de errores típicos que pueden aparecer durante el uso y mantenimiento del proyecto, con su diagnóstico y solución.

6.1 Errores de git

Síntoma	Causa probable	Solución
'git' no se reconoce como comando	Git no instalado o PATH no actualizado	Reinicia la ventana de cmd. Si persiste, reinstala Git for Windows marcando la opción de añadir al PATH.
fatal: not a git repository	Estás fuera de la carpeta del proyecto	Ejecuta cd %USERPROFILE%\vassal-osc antes de cualquier git.

Síntoma	Causa probable	Solución
error: failed to push some refs	Hay commits en remoto que no tienes localmente	Ejecuta git pull antes de git push.
Authentication failed / 403 al pushear	PAT caducado o incorrecto	Ve a https://github.com/settings/tokens y regenera el PAT. Ver sección 5.2.
CONFLICT (content): Merge conflict in X	Merge con cambios simultáneos en mismo archivo	Resuelve manualmente (ver sección 2.4) o aborta con git merge --abort.
Updates were rejected because the remote contains work	Repo remoto con commits que no tienes	git pull --rebase origin main y luego git push.
warning: LF will be replaced by CRLF	Solo un aviso (line endings)	Ignóralo. No es un error.
'cgit' no se reconoce...	Typo al escribir 'git'	Escribe el comando correctamente: git ...

6.2 Errores de npm

Síntoma	Causa probable	Solución
'npm' no se reconoce como comando	Node no instalado o PATH no actualizado	Reinicia cmd. Si persiste, reinstala Node.js LTS.
npm ERR! ENOENT durante npm install	Alguna dependencia no se descarga bien	Borra node_modules y package-lock.json y vuelve a npm install.
npm WARN deprecated	Solo avisos, no errores	Ignóralos. Mientras no aparezca npm ERR! al final, está bien.
npm WARN install-scripts (esbuild)	Aviso sobre postinstall de esbuild	Normalmente es solo informativo. Si el dev server falla, ejecuta npm install-scripts approve esbuild y npm rebuild esbuild.
Error: ENOENT: no such file or directory 'C:\C:\...'	Bug del gen-presets.mjs sin parchear	Aplica el parche descrito en sección 2.4: sustituir .pathname por fileURLToPath en scripts/gen-presets.mjs.
Cannot find module 'docx' (u otro)	Dependencia no instalada	Ejecuta npm install en la raíz del proyecto.
Vite no arranca: EADDRINUSE :::5173	Puerto 5173 ya ocupado por otro proceso	Cierra el otro proceso o arranca con npx vite --port 5174.

6.3 Errores del workflow GitHub Actions

Step que falla	Causa probable	Solución
actions/configure-pages → Not Found	Pages no activado con source GitHub Actions	Settings → Pages → Source: GitHub Actions. Re-run del workflow.
npm ci → error	package-lock.json corrupto o ausente	En local: rm package-lock.json && npm install. Commit del nuevo lock y push.
npm run build → TypeScript error	Error en código TS del emulador	Mira el log del step Build. Prueba en local con npm run build para reproducir.
upload-pages-artifact → size limit	El dist generado pesa > 10 GB (límite de Pages)	Impensable con este setup. Solo pasará con docenas de .vmod empaquetados.
deploy-pages → HttpError	Pages mal configurado o timeout	Re-run del workflow. Si persiste, mira Settings → Pages.

6.4 Errores de carga del módulo en el emulador

Síntoma en navegador	Causa probable	Solución
Pantalla en blanco al cargar módulo	Error JS en consola (F12)	Abre F12 → Console y mira el error. Suele ser un trait no soportado o una pieza corrupta.
Error 'Unknown trait' en consola	El emulador no implementa ese trait	Anótalo. Puedes (a) reportar bug al autor, (b) añadir soporte en src/, (c) editar buildFile.xml del .vmod para quitar el trait.
No se ven las piezas en la paleta	Problema con la decodificación de pieces o prototypes	F12 → Console. Suele venir con stack trace del parser.
Mapa se ve pero hex grid desalineado	Bug en el rendering de grid	Reporta issue al autor con screenshot.
Botón de menú no responde al click	Probablemente dependa de clase Java	Sin Java no se puede ejecutar. Juega la función manualmente.
.vmod tarda una eternidad en cargar	Módulo grande (≥ 45 MB) o conexión lenta	Normal. La primera carga cachea; las siguientes van por SW.

6.5 Errores al guardar/cargar .vsav

Síntoma	Causa probable	Solución
iOS no deja seleccionar el .vsav (archivo en gris)	Filtro accept del input file restrictivo	Verifica que el parche de src/app.ts está aplicado (comentar la línea input.accept). Si sigue pasando, renombra el .vsav a .zip como workaround.
El .vsav carga pero la partida está corrupta	Versión del módulo cambió desde que se guardó	Carga la misma versión del .vmod con la que se guardó el .vsav.
El .vsav no carga tras un cambio de versión del emulador	Cambios en el formato interno del save	Incompatible con .vsav viejos. Vuelve a la versión anterior del emulador o empieza partida nueva.
El botón Save no responde	Error JS en consola	F12 → Console. Suele ser un problema con el modelo del juego en memoria.
El .vsav se guarda pero no se ve en Descargas	Se guardó en otra ubicación	Mira en app Archivos → En mi iPad → Safari, o iCloud Drive.

6.6 Errores en el iPad

Síntoma en iPad	Causa probable	Solución
Sitio no carga - 404 GitHub Pages	URL mal escrita (mayúsculas)	Verifica que la URL es https://javiarq.github.io/vassal-ocs/ todo en minúsculas.
Sitio no carga - sin error claro	Problema de red o DNS	Prueba otra web. Verifica WiFi. Prueba con hotspot del móvil.
App abre pero se ve versión vieja	SW cacheando versión antigua	Ver sección 4.4 (cómo forzar refresco).
Safari se cierra al cargar módulo grande	Memoria agotada en iPad antiguo	Cierra otras apps. Considera cargar el módulo en un iPad con más RAM.
No aparece el icono de Añadir a pantalla de inicio	Opción de Safari mal buscada	Botón Compartir (cuadrado con flecha arriba) → scroll → Añadir a pantalla de inicio.
Tapping icono abre Safari en vez de pantalla completa	Modo PWA no activado	Asegúrate de añadir desde Safari (no desde Chrome) y de usar Añadir a pantalla de inicio, no Añadir a marcadores.
Errores de certificado al abrir el sitio	Reloj del iPad mal puesto	Ajustes → General → Fecha y hora → Activa Ajustar automáticamente.

Síntoma en iPad	Causa probable	Solución
No se puede cargar .vmod desde Archivos	Explorador de archivos no abre	Cierra la app y vuelve a abrirla. Verifica que el .vmod está realmente en Descargas.

7. Cheat sheet — comandos rápidos

Tabla de referencia con los comandos más usados en el mantenimiento del proyecto. Imprimible o consultable de un vistazo.

7.1 Setup inicial (una sola vez)

Tabla 1 — Comandos de setup inicial

Acción	Comando
Entrar al proyecto	cd %USERPROFILE%\vassal-ocs
Añadir remote upstream	git remote add upstream https://github.com/TyroneSlothrop/vassal-ts-emulator.git
Configurar identidad git	git config --global user.name "Tu Nombre"
Configurar email git	git config --global user.email tu@email.com

7.2 Rutinas habituales

Tabla 2 — Comandos de mantenimiento

Acción	Comando
Sincronizar con upstream	git fetch upstream && git merge upstream/main && git push
Generar presets	npm run presets
Arrancar dev server	npm run dev
Build producción local	npm run build
Ver status de git	git status
Ver qué ha cambiado	git diff
Ver remotes	git remote -v
Ver log reciente	git log --oneline -10
Deshacer git add	git reset

Acción	Comando
Deshacer git add de un archivo	git reset <archivo>
Abortar un merge en conflicto	git merge --abort
Bump cache version SW	Editar public/sw.js → cambiar CACHE → git add → commit → push

7.3 URLs importantes

Tabla 3 — URLs de referencia

Concepto	URL
Repositorio GitHub	https://github.com/JaviARQ/vassal-ocs
Sitio Pages (jugar)	https://javiarq.github.io/vassal-ocs/
Workflow Actions	https://github.com/JaviARQ/vassal-ocs/actions
Configuración Pages	https://github.com/JaviARQ/vassal-ocs/settings/pages
Repositorio original (upstream)	https://github.com/TyroneSlothrop/vassal-ts-emulator
Biblioteca de módulos VASSAL	https://vassalengine.org/library
Personal Access Tokens	https://github.com/settings/tokens

7.4 Archivos clave del proyecto

Tabla 4 — Archivos importantes y su función

Archivo	Función
package.json	Dependencias y scripts npm (presets, dev, build)
vite.config.ts	Configuración del bundler. base: "./" para subdirectorio
.github/workflows/deploy.yml	Workflow de GitHub Actions: build + deploy a Pages
public/sw.js	Service worker (offline cache)
public/sw-register.js	Registro del SW desde index.html
index.html	Entrada del emulador; carga el SW
scripts/gen-presets.mjs	Genera presets.json (PARCHEADO para Windows)
src/app.ts	Código principal del emulador (PARCHEADO para iOS Safari .vsav)
public/modules/presets.json	Índice de módulos empaquetados (vacío por defecto)

Archivo	Función
.gitignore	Excluye node_modules, dist, etc.

7.5 Flujo de juego offline (resumen)

Tabla 5 — Pasos para jugar offline en el iPad

Paso	Acción
1	Descargar .vmod desde vassalengine.org en Safari del iPad (online, una vez por módulo)
2	Abrir la app VASSAL desde el icono de pantalla de inicio
3	Botón Load module from file → Descargas → seleccionar .vmod
4	Jugar. Para guardar: botón Save (baja .vsav a Descargas)
5	Para cargar partida: botón Load partida → seleccionar .vsav
6	El módulo debe estar cargado ANTES de cargar un .vsav

8. Glosario

Definiciones de los términos técnicos que aparecen en este documento y en el día a día del proyecto.

8.1 Git y GitHub

Término	Definición
git	Sistema de control de versiones distribuido. Registra cambios en archivos y permite trabajar con varios repositorios remotos.
commit	Conjunto de cambios registrados en git con un mensaje descriptivo.
push	Envía tus commits locales al repositorio remoto (origin o el que sea).
pull	Trae commits del repositorio remoto a tu copia local.
fetch	Como pull pero sin fusionar. Solo descarga los cambios; tú decides cuándo aplicarlos.
merge	Fusiona dos conjuntos de cambios en uno. Se usa para traer cambios de upstream a tu main.
clone	Crea una copia local completa de un repositorio remoto.
fork	Copia de un repo en tu cuenta de GitHub. Se hace desde el botón Fork en la web.

Término	Definición
remote	Puntero a un repositorio remoto. Por defecto al clonar se crea origin.
origin	Remote apuntando a TU repo (github.com/JaviARQ/vassal-ocs).
upstream	Remote apuntando al repo original (github.com/TyroneSlothrop/vassal-ts-emulator).
PAT (Personal Access Token)	Contraseña larga generada en GitHub para autenticar push desde la línea de comandos. Sustituye a la contraseña normal desde 2021.
GCM (Git Credential Manager)	Herramienta incluida en Git for Windows que gestiona autenticación vía navegador OAuth.
main	Nombre de la rama principal del proyecto. Antes se llamaba master.
PR (Pull Request)	Propuesta de cambio al repo de otra persona. Se abre desde la web de GitHub.

8.2 Node.js, npm y Vite

Término	Definición
Node.js	Entorno de ejecución de JavaScript fuera del navegador. Necesario para construir el proyecto.
npm	Gestor de paquetes de Node. Instala dependencias (npm install) y ejecuta scripts (npm run X).
package.json	Archivo que lista dependencias y scripts del proyecto.
package-lock.json	Snapshot de las versiones exactas instaladas. Garantiza builds reproducibles.
node_modules/	Carpeta con todas las dependencias instaladas. No se sube a git (reconstruida con npm install).
npm ci	Instalación limpia basada en package-lock.json. La usa el workflow de Actions.
Vite	Bundler moderno para proyectos web. Compila TypeScript y genera dist/ optimizado para producción.
dev server	Servidor local de desarrollo (npm run dev). Sirve el sitio en http://localhost:5173.
dist/	Carpeta con el build de producción. Generada por npm run build, no se sube a git.
TypeScript (TS)	Superset de JavaScript con tipos estáticos. El emulador está escrito en TS.

8.3 VASSAL y OCS

Término	Definición
VASSAL	Engine open-source para jugar juegos de tablero y cartas online. Originalmente en Java.
.vmod	Archivo de módulo VASSAL. Es un ZIP renombrado que contiene buildFile.xml, moduledata y recursos.
buildFile.xml	XML dentro del .vmod que define piezas, tableros, menús, traits. Lo lee el emulador TS.
moduledata	Archivo dentro del .vmod con metadatos: nombre, versión, autor.
trait	Comportamiento de una pieza (rotación, etiqueta, marca de suministro, etc.). Cada pieza puede tener varios traits apilados.
Java personalizado	Clases .class dentro de un .vmod que implementan lógica avanzada. El emulador TS no las ejecuta.
OCS	Operational Combat Series. Serie de wargames operacionales publicada por MMP (Multi-Man Publishing).
MMP	Multi-Man Publishing. Editorial que publica OCS y otros wargames.
preset	Entrada en presets.json que describe un módulo cargable desde el selector del emulador.
.vsav	Archivo de partida guardada VASSAL. Compatible entre el emulador TS y VASSAL Java de escritorio. Es un ZIP.
Load module from file	Botón del emulador que permite cargar un .vmod desde el sistema de archivos local (no desde presets).

8.4 Service worker y PWA

Término	Definición
Service worker (SW)	Script JavaScript que el navegador instala en segundo plano y que intercepta peticiones de red para gestionar caché offline.
PWA (Progressive Web App)	Aplicación web que, con un SW y un manifest, se comporta como una app nativa: icono en pantalla de inicio, pantalla completa, offline.
cache	Almacenamiento local de assets (HTML, JS, .vmod). El SW lo gestiona.
cache version	Etiqueta única (vassal-ocs-v1) en el SW. Al cambiarla, se invalida la caché vieja y se descarga todo de nuevo.
network-first	Estrategia: probar red primero (versión nueva); si falla, servir cache.
stale-while-revalidate	Estrategia: servir cache al instante y descargar versión nueva en segundo plano para la próxima visita.

Término	Definición
skipWaiting()	Método del SW que activa la nueva versión inmediatamente sin esperar a que se cierren las pestañas.
clients.claim()	Método del SW que toma control de las pestañas abiertas inmediatamente al activarse.

8.5 GitHub Pages y GitHub Actions

Término	Definición
GitHub Pages	Servicio gratuito de hosting estático integrado en GitHub. Sirve sitios web desde repos.
GitHub Actions	Sistema de CI/CD integrado en GitHub. Ejecuta workflows (workflows YAML) en respuesta a eventos (push, PR, etc.).
workflow	Archivo YAML en .github/workflows/ que define un job: pasos, entorno, permisos.
runner	Servidor donde se ejecuta el workflow (ubuntu-latest, windows-latest, etc.).
step	Cada paso dentro de un job: checkout, setup-node, npm ci, etc.
artifact	Colección de archivos producida por un workflow y subida a GitHub para uso posterior (p.ej. dist/ que se deploya a Pages).
Source: GitHub Actions	Configuración de Pages que indica que el sitio se publica desde un workflow, no desde una rama.